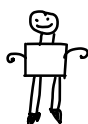


C

Programming

Language

Noted By



Contents

Chapter-1:	5
Basic syntax, variables and data types	5
1.1 History of Programming and Computer:	5
1.2 History of C programming Language:	5
1.3 First C Program:	5
1.2 Comments:	7
1.3 Variables and Data Types:	8
1.4 Data Types Limitations in C:	10
1.5 How to Take Input in C:	10
1.6 pre and post increment:.....	11
1.7 Operator Precedence in C:	12
Chapter 2:.....	13
Operators, Conditional Statements(if-else)	13
2.1 Arithmetic Operators: +, -, *, /, %.....	13
2.2 Relational Operators: >, <, >=, <=, ==, !=.....	14
2.3 Logical Operators: &&, , !	14
2.4 If Else:	15
2.5 If Else Ladder:	15
2.6 Nested If Else:.....	16
Chapter 3: Loop.....	17
3.1 For Loop:	17
3.2 Break Statement:	20
3.3 continue Statement:	20
3.4 While and Do-While Loop:	21
3.5 Nested Loop:	23
Chapter 4: Introduction To Array	30
4.1 What is Array:.....	30
4.2 Array Input and Output:.....	31
4.3 Printing Reverse of an Array:.....	35
4.4 Reverse array Element (Two pointers technique):	36
4.5 Selection sort:	39
4.6 Sum of an Array:.....	43
4.7 Counting array:.....	44
4.8 sum of two value equal x:	47
4.9 insert element in array:.....	48

4.10 Remove element from an array:	50
4.11 Array Concatenation:	52
Chapter 5: 2D Array.....	54
5.1 What is 2D Array:	54
5.2 2d array input and output:.....	55
5.3 How to print specific row and column in 2D Array:	56
5.4 Different type of Matrix:	57
Chapter 6: introduction to string	65
6.1 what is string:.....	65
6.2 string input and output:.....	67
6.3 Length of a string:	70
6.4 string copy:.....	72
6.5 string lexicographical comparison:.....	73
6.6 string concatenation:	76
6.7 counting or frequency String (Problems):.....	78
Chapter 7: Function.....	81
7.1 what is function:	81
7.2 return + parameter:	84
7.3 return + no parameter:	85
7.4 No return + parameter:.....	87
7.5 no return + no parameter:	87
7.6 useful built-in function:.....	88
7.7 scopes in c:	89
Chapter 8: Recursion.....	91
8.1 call Stack:.....	91
8.2 What is Recursion:	94
8.3 Print 1 to 5 using recursion:	95
8.4 Print 5 to 1 using recursion:	95
8.5 printing array using recursion:	96
8.6 Length of a string using recursion:	97
Chapter 9: Pointer	98
9.1 Pointers:	98
9.2 Call by Value or pass by value:	100
9.3 Call by reference or passed by reference (Pointer Dereferencing Technique):.....	100
9.4 Relation Between array and pointer:	101
9.5 How to pass array into a function:	103

9.6 How to pass string into a function:	104
---	-----

Chapter-1:

Basic syntax, variables and data types

1.1 History of Programming and Computer:

- 1837 = Charles Babbage conceptualized and designed the Analytical Engine, the first mechanical general-purpose computer.
- 1843 = Ada Lovelace wrote an algorithm for the Analytical Engine. Her algorithms introduced foundational programming concepts such as loops, conditional branching, and the idea of data storage, earning her recognition as the world's first computer programmer.

1.2 History of C programming Language:

- 1958 (ALGOL) = ALGOL (Algorithmic Language) was developed as a structured programming language, influencing the development of many modern languages, including C, particularly in concepts like block structure and nested functions.
- 1966 (BCPL): BCPL (Basic Combined Programming Language) was developed for system-level programming. It influenced the creation of B, which was a direct precursor to C.
- 1970 (B) = B is a simpler version of BCPL. It was created at Bell Labs for early UNIX OS development but lacked essential data types and structures, which limited its ability to handle more complex software development.
- 1972 (C): C was developed by Dennis Ritchie at Bell Labs to rewrite the UNIX operating system. Its design made it portable across different hardware platforms, and it became one of the most influential languages in software development.

1.3 First C Program:

```
#include <stdio.h>
int main()
{
    printf("Hello World");

    return 0;
}
```

After writing the program open vs code terminal and type:

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ gcc hello.c -o hello

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
Hello, World!
```

Behind the code:

1. Directives: Directives are special instructions for the preprocessor that start with a # symbol.

Some common directives:

- #include = Includes the contents of another file into the current file, often used to include header files.
- #define = Define a macro.

2. Macros: Macros are special types of directives that allow you to define constants or reusable code snippets that defined using the #define directives.

Example:

```
#include <stdio.h>
#define PI 3.1416

int main()
{
    printf("Value of PI: %.4f", PI);

    return 0;
}
```

```
$ ./hello
Value of PI: 3.1416
conta@Tamin MINGW64 ~/Downloads/C Programming
```

3. Preprocessor: Preprocessor is a tool or step in the compilation process that handles directives and macros before the actual compilation begins.

Key features of the preprocessor:

- It runs automatically before the compiler.
- It removes comments and expands macros.

What the preprocessor does:

- Reads the directives (Lines starting with #) in the source code.
- Modifies the code as per the directives and expands macros.
- Produces the preprocessed source code that is then passed to the compiler.

4. Header file: A header file in C is a file with a .h extension that contains reusable code, such as function declaration, constants, variables, and data types. You can include it in your program using the #include directive.

Some Common header files in c:

- stdio.h – Standard Input/Output functions
- stdlib.h – Memory allocation, process control, conversions
- math.h – Mathematical functions
- string.h – String handling functions

- stdbool.h – Boolean data type (true, false, bool)

5. **main function(){}:** The main() function is the entry point of every C program. You must include the main() function once in your code for the program to work. No matter where you write the main() function, when the C program runs, it finds the main() function first and starts executing from there.

6. **printf() Function:** The printf() is a predefined function; If we want to use this function we have to include our standard input and output header or library file (stdio.h).

1.4 Comments:



```
#include <stdio.h>

int main()
{
    // This is a single line comment
    /*
        This is a
        Multiline
        Comment
    */

    return 0;
}
```

Some Languages and Markup languages comments:

HTML: <!--HTML Comments -->

CSS: /*CSS Comments*/

JS: // JS Single Line Comments

/*JS Multi

Lines Comments*/

1.5 Variables and Data Types:

Data Types:

1. int: - 4, -3, -2, -1, 0, 1, 2, 3, 4 ---%d (Format Specifier)
2. long long int: - 4, -3, -2, -1, 0, 1, 2, 3, 4 ---%lld
3. float: -4.53, -3.45, 1.5, 3.1416 ---%f
4. double: -4.53, -3.45, 1.5, 3.1416 ---%lf
5. char: '1', '5', 'A', '@', --- %c
6. bool: true(1) or false(0)

Variable: Who store data types.

Variable Declaration:

```
int a;
```

Variable initialization:

```
int a = 30;
```

Variable assignment:

```
a = 40;
```

Note: initialization gives a variable its first value, while assignment gives a variable a new value after it has been initialized.

Example:

```
#include <stdio.h>
#include <stdbool.h>

int main()
{
    int myInt = 2147483647;
    long long int myLongLongInt = 9223372036854775807;
    float myFloat = 1.123456;
    double myDouble = 1.12345678912345;
    char myChar = 'a';
    bool myBool = true;

    printf("myInt: %d\n", myInt);
    printf("myLongLongInt: %lld\n", myLongLongInt);
    printf("myFloat: %f\n", myFloat);
    printf("myFloat: %.2f\n", myFloat);
    printf("myDouble: %.15lf\n", myDouble);
    printf("myChar: %c\n", myChar);
    printf("myBool: %d\n", myBool);

    return 0;
```

```
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
● $ ./hello
myInt: 2147483647
myLongLongInt: 9223372036854775807
myFloat: 1.123456
myFloat: 1.12
myDouble: 1.123456789123450
myChar: a
myBool: 1
```

1.6 Data Types Limitations in C:

int = 10^9 = it allows up to approximately 10 digits

long long int = 10^{18} = it allows up to approximately 19 digits

float = 10^6 = it allows up to approximately 7 digits precision (1.123456 = 7 digits)

double = 10^{14} = it allows up to approximately 15 digits precision

char = -128 to 127

1.7 How to Take Input in C:

```
#include <stdio.h>
int main()
{
    int myInt;
    float myFloat;
    char myChar;

    scanf("%d", &myInt); // & = address of operator or ampersand operator
    scanf("%f", &myFloat);
    scanf(" %c", &myChar); //The space before %c to consume any leftover
    newline character

    printf("Integer: %d\n", myInt);
    printf("Float: %f\n", myFloat);
    printf("Float: %.2f\n", myFloat);
    printf("Character: %c\n", myChar);

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
10
10.10
T
Integer: 10
Float: 10.100000
Float: 10.10
Character: T
```

Some common input and output function:

1. `scanf()` -- `printf()`
2. `getchar()` -- `putchar()` = `getchar()` reads a single character, and `putchar()` prints a single character.

Code:

```
#include <stdio.h>
int main()
{
    char myChar;
    myChar = getchar();
    putchar(myChar); // Output the character read from input

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
T
T
```

Note:

We cannot directly call the `getchar()` function. Instead, we must assign the `getchar()` function to a variable.

We are not allowed to add any additional text inside the `putchar()` function and The `putchar()` function must strictly be used to print a single character.

1.8 pre and post increment:**Pre increment:**

```
#include <stdio.h>
int main()
{
    int i = 10;
    int x = ++i;
    // ----->

    printf("x = %d\n", x);
    printf("i = %d", i);

    return 0;
}
```

```
conta@tamim MINGW64 ~/Downloads/C Programming
$ ./hello
x = 11
i = 11
```

Note: Here, `i` is incremented to 11 first, and then this new value is assigned to `x`. Both `i` and `x` are 11 after this operation.

Post Increment:

```
#include <stdio.h>
int main()
{
```

```

int i = 10;
int x = i++;
// ----->

printf("x = %d\n", x);
printf("i = %d", i);

return 0;
}

```

```

$ ./hello
x = 10
i = 11

```

Note: First, the value of i (which is 10) is assigned to the variable x. After that, i is incremented, so i becomes 11.

Note:

Pre-increment (++i): First increments the value of i, then assigns it.

Post-increment (i++): First assigns the value, then increments i.

1.9 Operator Precedence in C:

Precedence Level	Operators	Associativity	Description
1 (Highest)	()	Left to Right	Parentheses (used for grouping)
2	*, /, %	Left to Right	Multiplication, Division, Modulus
3 (Lowest)	+, -	Left to Right	Addition, Subtraction

So, if you write an expression like:

```
int result = 10 + 5 - 2 / 2 * 3;
```

$10 + 5 - 1 * 3$

$10 + 5 - 3$

$15 - 3$

12

Final result = 12

Chapter 2:

Operators, Conditional Statements(if-else)

2.1 Arithmetic Operators: +, -, *, /, %

```
#include <stdio.h>

int main()
{
    int a = 10;
    int b = 50;

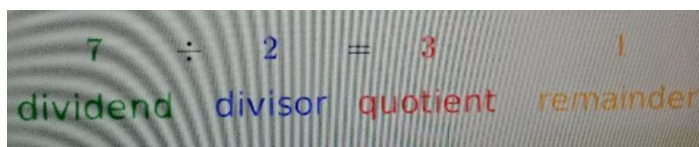
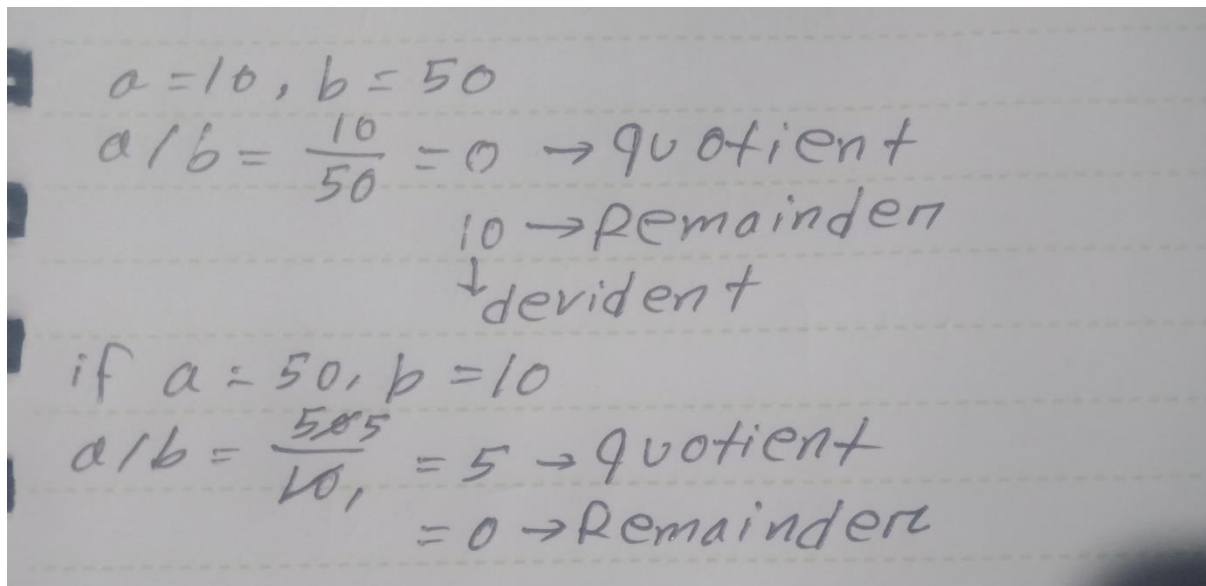
    int quotient1 = a / b;
    int remainder1 = a % b;

    int quotient2 = b / a;
    int remainder2 = b % a;

    printf("%d / %d = %d\n", a, b, quotient1);
    printf("%d %% %d = %d\n", a, b, remainder1);
    printf("%d / %d = %d\n", b, a, quotient2);
    printf("%d %% %d = %d\n", b, a, remainder2);

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
10 / 50 = 0
10 % 50 = 10
50 / 10 = 5
50 % 10 = 0
```



2.2 Relational Operators: >, <, >=, <=, ==, !=

2.3 Logical Operators: &&, ||, !

Logical not(!):

Syntax:

```
!Condition
```

```
// returns true if the conditions is false
// else returns false
```

Note: The logical NOT (!) operator understands two things: true (non-zero) and false (zero).

- If the value is true (non-zero), ! turns it into false (0).
- If the value is false (zero), ! turns it into true (1).

```
#include <stdio.h>
int main()
{
    int x = 5; // x = true
    if (!x)
        printf("X is zero\n");
    else
        printf("X is not zero\n");
}
```

```

x = 0; // x = false
if (!x)
    printf("X is zero\n");
else
    printf("X is not zero");

return 0;
}

```

```

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
X is not zero
X is zero

```

2.4 If Else:

```

If(condition == true){
    Statement 1;
}
Else{
    Statement 2;
}

```

2.5 If Else Ladder:

```

If(condition == ture){
    Statement 1;
}
Else if(condition == ture){
    Statement 2;
}
Else if(condition == ture){
    Statemtn 3;
}
Else{
    Statement 4;
}

```

2.6 Nested If Else:

```
If(condition == ture){  
    Statement 1;  
    If(condition == ture){  
        Statement 2;  
    }  
    Else{  
        Statement 3;  
    }  
}  
Else{  
    Statement 4;  
}
```

Chapter 3: Loop

3.1 For Loop:

Syntax: for(initialization; condition; increment/decrement) {

Statement;

}

Example 1:

```
#include <stdio.h>

int main()
{
    for (int i = 1; i <= 3; i++)
    /*
        i += 1;
        i = i + 1;
    */
    {
        printf("I am Sorry\n", i);
    }

    return 0;
}
```

```
conta@Tamin MINGW64 ~/Downloads/C Programming
$ ./hello
I am Sorry
I am Sorry
I am Sorry
```

Example 2:

```
#include <stdio.h>

int main()
{
    for (int i = 1; i <= 3; i++)
    {
        printf("%d\n", i);
    }

    return 0;
}
```

```

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
1
2
3

```

Example 3:

```

#include <stdio.h>

int main()
{
    for (int i = 1; i <= 20; i = i + 2)
    {
        printf("%d\n", i);
    }

    return 0;
}

```

```

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
1 3 5 7 9 11 13 15 17 19

```

Example 4: Sum Form 1 to N

```

#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    int sum = n * (n + 1) / 2;

    printf("%d", sum);

    return 0;
}

```

OR

```

#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int sum = 0;

    for (int i = 1; i <= n; i++)
    {

```

```

        sum += i;
        // sum = sum + i;
    }
    printf("%d", sum);

    return 0;
}

```

```

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
5
15

```

Example 5:

```

#include <stdio.h>

int main()
{
    for (int i = 1; i <= 10; i++)
    {
        if (i % 2 == 0)
        {
            printf("%d = even\n", i);
        }
        else
        {
            printf("%d = odd\n", i);
        }
    }

    return 0;
}

```

```

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
1 = odd
2 = even
3 = odd
4 = even
5 = odd
6 = even
7 = odd
8 = even
9 = odd
10 = even

```

3.2 Break Statement:

```
#include <stdio.h>

int main()
{
    for (int i = 1; i <= 10; i++)
    {
        if (i == 5)
        {
            break;
        }
        printf("%d ", i);
    }

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
1 2 3 4
```

3.3 continue Statement:

```
#include <stdio.h>

int main()
{
    for (int i = 1; i <= 10; i++)
    {
        if (i == 5)
        {
            continue;
        }
        printf("%d ", i);
    }

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
1 2 3 4 6 7 8 9 10
conta@Tamim MINGW64 ~/Downloads/C Programming
```

3.4 While and Do-While Loop:

While Loop:

Syntax:

Initialization;

```
While(condition){
    Statement;
    Increment/decrement;
}
```

Example:

```
#include <stdio.h>

int main()
{
    int i = 1;
    while (i <= 10)
    {
        printf("%d ", i);
        i++;
    }

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
1 2 3 4 5 6 7 8 9 10
conta@Tamim MINGW64 ~/Downloads/C Programming
```

Do while:

Syntax:

Initialization;

```
do{
    Statement;
    Increment/ decrement;
}
```

While(condition);

Example:

```
#include <stdio.h>
```

```
int main()
{
    int i = 1;
    do
    {
        printf("%d ", i);
        i++;
    } while (i <= 10);

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
1 2 3 4 5 6 7 8 9 10
conta@Tamim MINGW64 ~/Downloads/C Programming
#
```

3.5 Nested Loop:

pattern 1:

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
4
*
**
***
****
```

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j <= i; j++)
        {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
4
****
***
**
*
```

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        for (int j = n; j > i; j--)
        {
            printf("*");
        }
    }
}
```

```

    }
    printf("\n");
}
return 0;
}

```

pattern 2:

```

$ ./hello
4
1
12
123
1234

```

```

#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j <= i; j++)
        {
            printf("%d", j + 1);
        }
        printf("\n");
    }

    return 0;
}

```

```

$ ./hello
4
1234
123
12
1

```

```

#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    for (int i = n; i > 0; i--)
    {
        for (int j = 0; j < i; j++)

```

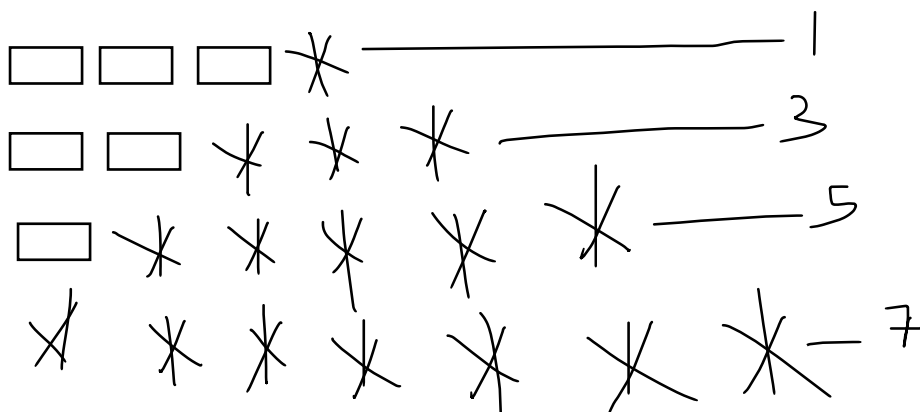
```

    {
        printf("%d", j + 1);
    }
    printf("\n");
}

return 0;
}

```

pattern 3:



```

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
4
 *
 ***
 *****
 *******

```

```

#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int space, star;
    space = n - 1;
    star = 1;

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < space; j++)
        {
            printf(" ");
        }
        for (int j = 0; j < star; j++)
        {

```

```

        printf("*");
    }
    printf("\n");
    space--;
    star += 2;
}

return 0;
}

```

```

$ ./hello
4
*****
****
***
**
*

```

```

#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int space, star;
    star = n * 2 - 1;
    space = 0;

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < space; j++)
        {
            printf(" ");
        }
        for (int j = 0; j < star; j++)
        {
            printf("*");
        }
    }
}

```

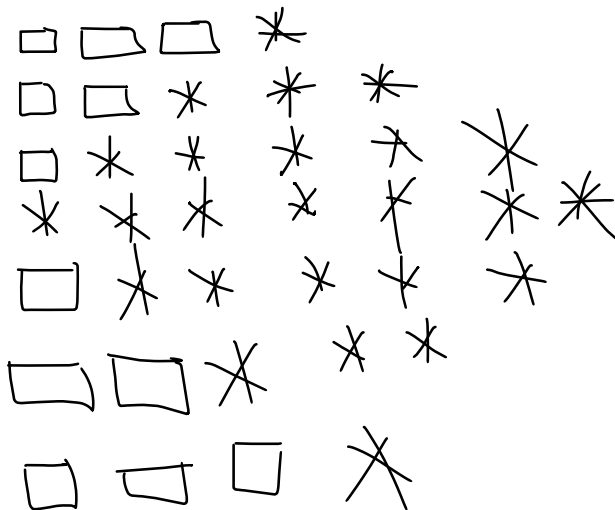
```

    }
    printf("\n");
    space++;
    star -= 2;
}

return 0;
}

```

Important:



```

4
*
***
*****
*****
*****
***
*

```

```

#include <stdio.h>

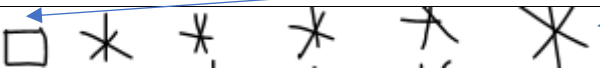
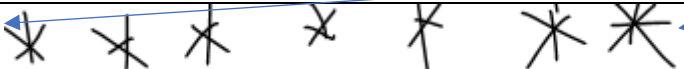
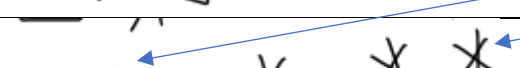
int main()
{
    int n;
    scanf("%d", &n);
    int space, star;
    space = n - 1;
    star = 1;

    for (int i = 0; i < n * 2 - 1; i++)
    {
        for (int j = 0; j < space; j++)
        {
            printf(" ");

```

```
    }  
    for (int j = 0; j < star; j++)  
    {  
        printf("*");  
    }  
    printf("\n");  
    if (i < n - 1)  
    {  
        space--;  
        star += 2;  
    }  
    else  
    {  
        space++;  
        star -= 2;  
    }  
}  
  
return 0;  
}
```

Explanation:

i	space	star	output	condition	Updated space	Updated star
0	3	1		if(0 < 3)	2	3
1	2	3		if(1 < 3)	1	5
2	1	5		if(2 < 3)	0	7
3	0	7		else	1	5
4	1	5		else	2	3
5	2	3		else	3	1
6	3	1		else	4	-1

Chapter 4: Introduction To Array

4.1 What is Array:

An array is a variable that can store multiple values in same data types.

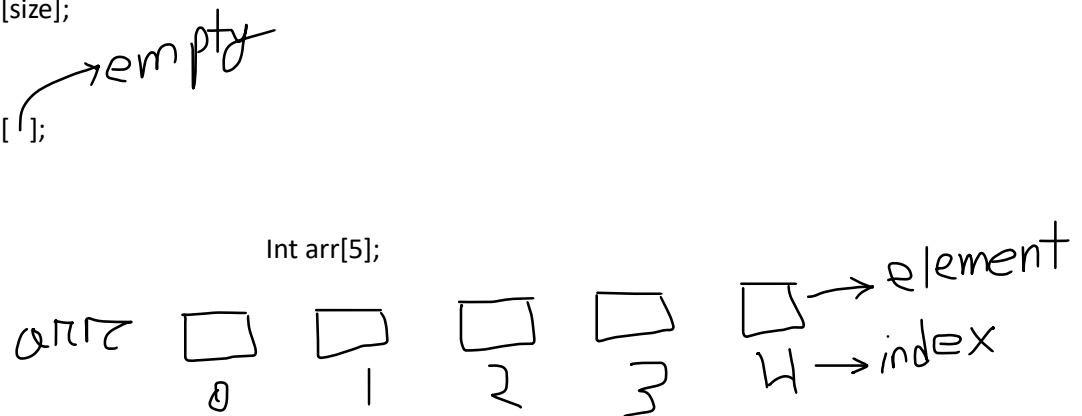
Array Declaration:

Syntax:

Data_type array_name [size];

Or

Data_type array_name [];



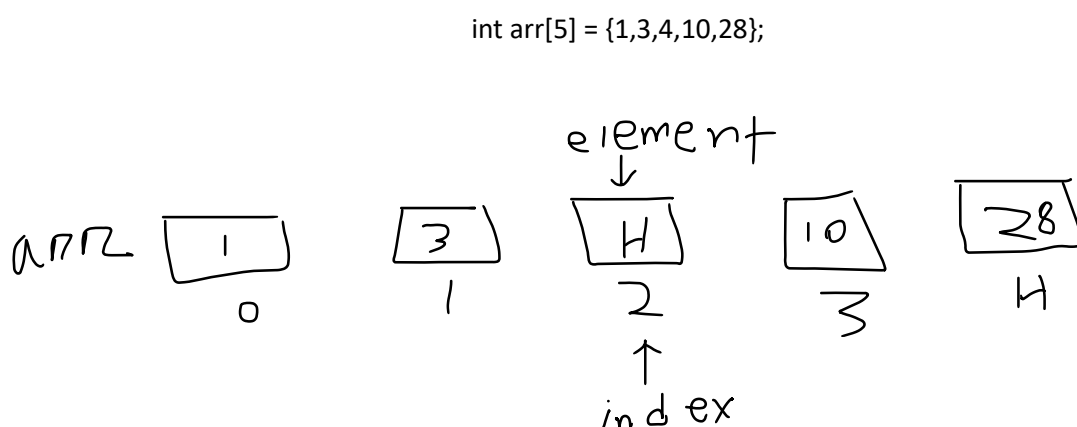
Array Initialization:

Syntax:

data_types array_name[size] = {value1, value2, value3.....};

Or

data_types array_name[] = {value1, value2, value3.....};



Array Assignment:

```
#include <stdio.h>

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};
    arr[0] = 100;
    arr[1] = 200;

    printf("%d %d %d %d %d", arr[0], arr[1], arr[2], arr[3], arr[4]);

    return 0;
}
```

```
conta@Tamil MINGW64 ~/Downloads/C Programming
$ ./hello
100 200 30 40 50
conta@Tamil MINGW64 ~/Downloads/C Programming
```

4.2 Array Input and Output:

Example-1:

```
#include <stdio.h>

int main()
{
    int arr[5];

    arr[0] = 10;
    arr[1] = 20;
    arr[2] = 30;
    arr[3] = 40;
    arr[4] = 50;

    printf("%d %d %d %d %d", arr[0], arr[1], arr[2], arr[3], arr[4]);

    return 0;
}
```

```
conta@Tamil MINGW64 ~/Downloads/C Programming
$ ./hello
10 20 30 40 50
conta@Tamil MINGW64 ~/Downloads/C Programming
```

Example-2:

```
#include <stdio.h>

int main()
{
    int arr[5];
    scanf("%d %d %d %d %d", &arr[0], &arr[1], &arr[2], &arr[3], &arr[4]);

    printf("%d %d %d %d %d", arr[0], arr[1], arr[2], arr[3], arr[4]);

    return 0;
}
```

```
conta@Tamin MINGW64 ~/Downloads/C Programming
$ ./hello
10 20 30 40 50
10 20 30 40 50
```

Example-3:

```
#include <stdio.h>

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

```
conta@Tamin MINGW64 ~/Downloads/C Programming
$ ./hello
10 20 30 40 50
```

Example- 4: (We will use)

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

```
conta@Tamil MINGW64 ~/Downloads/C Programming
$ ./hello
5
10 20 30 40 50
10 20 30 40 50
```

Example-5:

```
#include <stdio.h>

int main()
{
    int arr[5] = {100};
    for (int i = 0; i <= 4; i++)
    {
        printf("%d\n", arr[i]);
    }

    return 0;
}
```

```
conta@Tamil MINGW64 ~/Downloads/C Programming
$ ./hello
100
0
0
0
0
```

Example-6:

```
#include <stdio.h>

int main()
{
    int arr[5] = {};
    for (int i = 0; i < 5; i++)
    {
        printf("%d\n", arr[i]);
    }

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
0
0
0
0
0
```

Note: This May output garbage values (random numbers) because the array elements are not initialized. But if we don't want garbage values and need to initialize the entire array to 0, we can simply initialize the first element to 0. This will automatically set all the elements of the array to 0.

```
#include <stdio.h>

int main()
{
    int arr[5] = {0};
    for (int i = 0; i < 5; i++)
    {
        printf("%d\n", arr[i]);
    }

    return 0;
}
```

```
$ ./hello
0
0
0
0
0
```

Example-8:

```
#include <stdio.h>

int main()
{
    int arr[] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++)
    {
        printf("%d\n", arr[i]);
    }

    return 0;
}
```

```
$ ./hello
10
20
30
40
50
```

4.3 Printing Reverse of an Array:

```
contagiamim@MINGW64 ~/Downloads/C Programming
$ ./hello
4
10 20 30 40
40 30 20 10
```

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    for (int i = n - 1; i >= 0; i--)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

4.4 Reverse array Element (Two pointers technique):



Program 1:

```
#include <stdio.h>

int main()
{
    int arr[3] = {10, 20, 30};
    int temp = arr[0];
    arr[0] = arr[2];
    arr[2] = temp;

    printf("%d %d %d", arr[0], arr[1], arr[2]);

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
30 20 10
```

Program 2:

10	20	30	40	50
0	1	2	3	4

Output:

50	40	30	20	10
0	1	2	3	4

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int i = 0, j = n - 1;

    while (i < j)
    {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
        i++;
        j--;
    }

    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
5
10 20 30 40 50
50 40 30 20 10
conta@Tamim MINGW64 ~/Downloads/C Programming
```

Explanation:

Initial Array:

10	20	30	40	50
0	1	2	3	4

Step 1:

```
int i = 0, j = n - 1; // i = 0; j = n - 1 = 5 - 1 = 4
```

```
while(i < j){ // i < j = 0 < 4 = true
```

```
    int temp = arr[i]; // temp = arr[0] = 10;
```

```
    arr[i] = arr[j]; // arr[0] = arr[4] ; arr[0] = 50;
```

50	20	30	40	50
0	1	2	3	4

```
    arr[j] = temp; // arr[4] = temp ; arr[4] = 10;
```

50	20	30	40	10
0	1	2	3	4

```
    i++;
```

```
    j--;
```

```
}
```

Step 2:

```
while(i < j){ // i < j = 1 < 3 = true
```

```
    int temp = arr[i]; // temp = arr[1] = 20;
```

```
    arr[i] = arr[j]; // arr[1] = arr[3] ; arr[1] = 40;
```

50	40	30	40	10
0	1	2	3	4

```
    arr[j] = temp; // arr[3] = temp ; arr[3] = 20;
```

50	40	30	20	10
0	1	2	3	4

```
    i++;
```

```
    j--;
```

```
}
```

4.5 Selection sort:

Selection Sort finds the smallest element in the array, and when it finds the smallest element, it swaps it with the first unsorted element.

Unsorted:

20	40	10	30
----	----	----	----

Sorted Ascending order (Smallest to Largest):

10	20	30	40
----	----	----	----

Sorted Descending order (Largest to Smallest):

40	30	20	10
----	----	----	----

Sorted Ascending order code:

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (arr[i] > arr[j])
            {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }

    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
}
```

```

    return 0;
}
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
4
20 40 10 30
10 20 30 40

```

Explanation:

Initial Array:

20	40	10	30
0	1	2	3

Step 1:

```

for(int i = 0; i < n - 1; i++){ // 0 < 3 = true;
    for(int j = i + 1; j < n; j++){ // 1 < 4 = true;
        if(arr[i] > arr[j]){ // arr[0] > arr[1]; 20 > 40 = false;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
}

```

Again, inner loop,

```

for(int j = i + 1; j < n; j++){ // j++ = 2; 2 < 4 = true
    if(arr[i] > arr[j]){ // arr[0] > arr[2]; 20 > 10 = true;
        int temp = arr[i]; // temp = 20;
        arr[i] = arr[j]; // arr[0] = arr[2]; arr[0] = 10;

```

10	40	10	30
0	1	2	3

```

        arr[j] = temp; // arr[2] = 20;

```

10	40	20	30
0	1	2	3

```

    }
}

```

Again, inner loop,

```

for(int j = i + 1; j < n; j++){ //j++ = 3; 3 < 4 = true;
    if(arr[i] > arr[j]){ // arr[0] > arr[3]; 10 > 30 = false;
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

```

Step 2:

```

for(int i = 0; i < n - 1; i++){ //i++ = 1; 1 < 3 = true;
    for(int j = i + 1; j < n; j++){ // j = 2; 2 < 4 = true;
        if(arr[i] > arr[j]){ // arr[1] > arr[2]; 40 > 20 = true;
            int temp = arr[i]; // temp = 40;
            arr[i] = arr[j]; // arr[1] = arr[2]; arr[1] = 20;

```

10	20	20	30
0	1	2	3

```

        arr[j] = temp; // arr[2] = 40;

```

10	20	40	30
0	1	2	3

```

    }
}
}

```

Again, inner loop,

```

for(int j = i + 1; j < n; j++){ // j = 3; 4 < 4 = false;
    if(arr[i] > arr[j]){ // arr[1] > arr[3]; 20 > 30 = false;
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

```

Step 3:

```
for(int i = 0; i < n - 1; i++){ // i = 2; 3 < 3 = true;
    for(int j = i + 1; j < n; j++){ // j = 2 + 1; 3 < 4 = true;
        if(arr[i] > arr[j]){ // arr[2] > arr[3]; 40 > 30 = true;
            int temp = arr[i]; // temp = 40;
            arr[i] = arr[j]; // arr[2] = arr[3]; arr[2] = 30;
```

10	20	30	30
0	1	2	3

```
arr[j] = temp; // arr[3] = 40;
```

10	20	30	40
0	1	2	3

```
    }
}
}
```

Sorted Descending order code:

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (arr[i] < arr[j])
            {
                int temp = arr[i];
                arr[i] = arr[j];
```

```

        arr[j] = temp;
    }
}

for (int i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}

return 0;
}

```

```

conta@Tamin MINGW64 ~/Downloads/C Programming
$ ./hello
4
20 40 10 30
40 30 20 10

```

4.6 Sum of an Array:

```

#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int sum = 0;
    for (int i = 0; i < n; i++)
    {
        sum += arr[i];
    }

    printf("%d", sum);

    return 0;
}

```

```

conta@Tamin MINGW64 ~/Downloads/C Programming
$ ./hello
4
1 2 3 4
10

```

4.7 Counting array:

Program 1:

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d Numbers between 0-5:\n", n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int cnt_0 = 0, cnt_1 = 0, cnt_2 = 0,
        cnt_3 = 0, cnt_4 = 0, cnt_5 = 0;
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == 0)
        {
            cnt_0++;
        }
        else if (arr[i] == 1)
        {
            cnt_1++;
        }
        if (arr[i] == 2)
        {
            cnt_2++;
        }
        if (arr[i] == 3)
        {
            cnt_3++;
        }
        if (arr[i] == 4)
        {
            cnt_4++;
        }
        if (arr[i] == 5)
        {
            cnt_5++;
        }
    }

    printf("0 = %d\n", cnt_0);
```

```

    printf("1 = %d\n", cnt_1);
    printf("2 = %d\n", cnt_2);
    printf("3 = %d\n", cnt_3);
    printf("4 = %d\n", cnt_4);
    printf("5 = %d\n", cnt_5);

    return 0;
}

```

```

$ ./hello
8
Enter 8 Numbers between 0-5:
1 1 0 1 1 3 2 2
0 = 1
1 = 4
2 = 2
3 = 1
4 = 0
5 = 0

```

Program 2:

```

#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d Numbers between 0-%d:\n", n, n - 1);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int cnt[6] = {0};
    for (int i = 0; i < n; i++)
    {
        cnt[arr[i]]++;
    }

    for (int i = 0; i < n; i++)
    {
        printf("%d = %d\n", i, cnt[i]);
    }

    return 0;
}

```

```
$ ./hello
6
Enter 6 Numbers between 0-5:
2 4 2 3 5 5
0 = 0
1 = 0
2 = 2
3 = 1
4 = 1
5 = 2
```

Explanation:

```
arr = {2, 4, 2, 3, 5, 5};
```

```
int cnt[7] = {0},
```

```
{0, 0, 0, 0, 0, 0, 0}
```

```
for(int i = 0; i < n; i++){
```

```
    cnt[arr[i]]++;
```

```
}
```

```
cnt[arr[0]]++;
```

```
cnt[2]++;
```

```
{0, 0, 1, 0, 0, 0, 0}
```

4.8 sum of two value equal x:

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int x;
    scanf("%d", &x);
    int flag = 0; // 0 = false, 1 = true
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (arr[i] + arr[j] == x)
            {
                flag = 1;
            }
        }
    }

    if (!flag) // flag == 0
    {
        printf("No");
    }
    else
    {
        printf("Yes");
    }

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
5
5 2 2 3 7
9
Yes
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
5
5 2 2 3 7
20
No
```

4.9 insert element in array:

Int arr[6] = {10,20,30,40,50};

10	20	30	40	50	
arr[0]	arr[1]	arr[2]	arr[3]	arr[4]	arr[5]

10	20	30	40	50	50
				Arr[5] = arr[4]	

10	20	30	40	40	50
				Arr[4] = arr[3]	

10	20	30	30	40	50
				Arr[3] = arr[2]	

10	20	20	30	40	50
				Arr[2] = arr[1]	

10	100	20	30	40	50
				Arr[1] = 100;	

Program 1:

```
#include <stdio.h>

int main()
{
    int arr[6] = {10, 20, 30, 40, 50}; // here 6 is the array size

    arr[5] = arr[4]; // here 5 is the array index
    arr[4] = arr[3];
    arr[3] = arr[2];
    arr[2] = arr[1];
    arr[1] = 100;

    for (int i = 0; i < 6; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

```

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
10 100 20 30 40 50

```

Program 2:

```

#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int arr[n + 1];
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int idx, ele;
    scanf("%d %d", &idx, &ele);
    for (int i = n; i > idx; i--)
    {
        arr[i] = arr[i - 1];
    }
    arr[idx] = ele;

    for (int i = 0; i <= n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}

```

```

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
5
10 20 30 40 50
1 100
10 100 20 30 40 50

```

4.10 Remove element from an array:

Int arr[5] = {10,20,30,40,50};

10	20	30	40	50
arr[0]	arr[1]	arr[2]	arr[3]	arr[4]

10	30	30	40	50
arr[1] = arr[2]				

10	30	40	40	50
arr[2] = arr[3]				

10	30	40	50	50
arr[3] = arr[4]				

Program 1:

```
#include <stdio.h>

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};
    arr[1] = arr[2];
    arr[2] = arr[3];
    arr[3] = arr[4];

    for (int i = 0; i < 5 - 1; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
10 30 40 50
```

Program 2:

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int idx;
    scanf("%d", &idx);

    for (int i = idx; i < n - 1; i++)
    {
        arr[i] = arr[i + 1];
    }

    for (int i = 0; i < n - 1; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
5
10 20 30 40 50
1
10 30 40 50
```

4.11 Array Concatenation:

Int a[5]

10	20	30	40	50
----	----	----	----	----

0 1 2 3 4

int b[2]

60	70
----	----

0 1

Int AB[7]

10	20	30	40	50	60	70
----	----	----	----	----	----	----

0 1 2 3 4 5 6

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);
    int a[n];
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    int m;
    scanf("%d", &m);
    int b[m];
    for (int i = 0; i < m; i++)
    {
        scanf("%d", &b[i]);
    }

    int ab[n + m];
    for (int i = 0; i < n; i++)
    {
        ab[i] = a[i];
    }
    for (int i = 0; i < m; i++)
    {
        ab[n + i] = b[i];
    }

    for (int i = 0; i < n + m; i++)
    {
        printf("%d ", ab[i]);
    }

    return 0;
}
```

```
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
5
10 20 30 40 50
2
60 70
10 20 30 40 50 60 70
conta@Tamim MINGW64 ~/Downloads/C Programming
```

Chapter 5: 2D Array

5.1 What is 2D Array:

A 2D array is essentially an array of arrays. The mathematical representation of a 2D array is a matrix.

Declaration:

A 2D array is declared by specifying the number of rows (number of arrays required) and columns (size of those arrays) in square brackets. For example, to declare a 2D array of integers with 3 rows and 4 columns, you write:

```
#include <stdio.h>

int main()
{
    int array[3][4];

    return 0;
}
```

Initialization:

```
#include <stdio.h>

int main()
{
    int arr[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12},
    };

    return 0;
}
```

Assignment:

```
#include <stdio.h>

int main()
{
    int arr[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12},
    };
}
```

```

    arr[0][1] = 50; // first array, second element

    return 0;
}

```

5.2 2d array input and output:

```

#include <stdio.h>

int main()
{
    int row, col;
    scanf("%d %d", &row, &col);

    int arr[row][col];

    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            scanf("%d", &arr[i][j]);
        }
    }

    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

```

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
3 4
1 2 3 4
5 6 7 8
9 10 11 12
1 2 3 4
5 6 7 8
9 10 11 12

```

5.3 How to print specific row and column in 2D Array:

In a 2D array, if we want to print a specific row, we have to run loop based on the column(array length) and if we have to print a specific column, then we have to run loop based on the rows(numbers of array).

```
#include <stdio.h>

int main()
{
    int row, col;
    scanf("%d %d", &row, &col);

    int arr[row][col];
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            scanf("%d", &arr[i][j]);
        }
    }

    int r;
    printf("Enter the specific row that you want to print: ");
    scanf("%d", &r);
    for (int i = 0; i < col; i++)
    {
        printf("%d ", arr[r][i]);
    }

    int c;
    printf("\nEnter the specific column that you want to print: ");
    scanf("%d", &c);
    for (int i = 0; i < row; i++)
    {
        printf("%d\n", arr[i][c]);
    }

    return 0;
}
```

```
$ ./hello
3 3
1 2 3
4 5 6
7 8 9
Enter the specific row that you want to print: 0
1 2 3
Enter the specific column that you want to print: 2
3
6
9
```

5.4 Different type of Matrix:

1. Zero Matrix (Null Matrix):

A matrix where all elements are zero.

Example: $\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$

```
#include <stdio.h>

int main()
{
    int row, col;
    scanf("%d %d", &row, &col);

    int arr[row][col];
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            scanf("%d", &arr[i][j]);
        }
    }

    int cnt = 0;
    int element = row * col;
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            if (arr[i][j] == 0)
            {
                cnt++;
            }
        }
    }

    if (element == cnt)
    {
        printf("Null Matrix");
    }
    else
    {
        printf("Not a Null Matrix");
    }

    return 0;
}
```

```

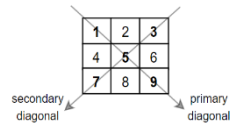
conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
2 2
0 0

```

2. Diagonal Matrix:

A square matrix where all the elements outside the Primary diagonal are zero.

Example: $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$



Note: Look at the diagonal Matrix index: $\begin{bmatrix} 00 & 01 & 02 \\ 10 & 11 & 12 \\ 20 & 21 & 22 \end{bmatrix}$

```

#include <stdio.h>

int main()
{
    int row, col;
    scanf("%d %d", &row, &col);

    int arr[row][col];
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            scanf("%d", &arr[i][j]);
        }
    }

    int flag = 1;
    // check if the matrix is square matrix or not
    if (row != col)
    {
        flag = 0;
    }
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            // if the row and column index are same that means we are now in
            the primary diagonal
            if (i == j)
            {
                continue;
            }
            if (arr[i][j] != 0)
            {
                flag = 0;
            }
        }
    }
}

```

```

    }
}

if (flag == 1)
{
    printf("Diagonal Matrix");
}
else
{
    printf("Not Diagonal Matrix");
}

return 0;
}

```

```

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
3 3
1 0 0
0 2 0
0 0 3
Diagonal Matrix

```

Note: we can further define diagonal matrix into two types

- i. **Primary Diagonal Matrix:** (Diagonal matrix and primary Diagonal Matrix are same thing).
- ii. **Secondary Diagonal Matrix:**

A diagonal matrix where all the elements outside the secondary diagonal are zero.

Example: $\begin{bmatrix} 0 & 0 & 1 \\ 0 & 2 & 0 \\ 3 & 0 & 0 \end{bmatrix}$

Note: Look at the secondary diagonal index: $\begin{bmatrix} 0 + 0 = 0 & 0 + 1 = 1 & 0 + 2 = 2 \\ 1 + 0 = 1 & 1 + 1 = 2 & 1 + 2 = 3 \\ 2 + 0 = 2 & 2 + 1 = 3 & 2 + 2 = 4 \end{bmatrix}$

```

#include <stdio.h>

int main()
{
    int row, col;
    scanf("%d %d", &row, &col);

    int arr[row][col];
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            scanf("%d", &arr[i][j]);

```

```

    }
}

int flag = 1;
// check if the matrix is square matrix or not
if (row != col)
{
    flag = 0;
}
for (int i = 0; i < row; i++)
{
    for (int j = 0; j < col; j++)
    {
        if (i + j == row - 1)
        {
            continue;
        }
        if (arr[i][j] != 0)
        {
            flag = 0;
        }
    }
}

if (flag == 1)
{
    printf("Secondary Diagonal Matrix");
}
else
{
    printf("Not secondary Diagonal Matrix");
}

return 0;
}

```

```

contag@famin MINGW64 ~/Downloads/C Programming
$ ./hello
3 3
0 0 1
0 2 0
3 0 0
Secondary Diagonal Matrix

```

3. Scalar Matrix:

A diagonal matrix (A square matrix where all the elements outside the Primary diagonal are zero) where all the diagonal elements are same.

Example: $\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$, $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, $\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$

```
#include <stdio.h>

int main()
{
    int row, col;
    scanf("%d %d", &row, &col);

    int arr[row][col];
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            scanf("%d", &arr[i][j]);
        }
    }

    int flag = 1;
    // check if the matrix is square matrix or not
    if (row != col)
    {
        flag = 0;
    }
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            if (i == j)
            {
                if (arr[i][j] != arr[0][0])
                {
                    flag = 0;
                }
            }
            else if (arr[i][j] != 0)
            {
                flag = 0;
            }
        }
    }

    if (flag == 1)
    {
        printf("Scalar Matrix");
    }
    else
    {
        printf("Not scalar Diagonal Matrix");
    }
}
```

```

    return 0;
}

```

```

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
2 2
2 0
0 2
Scalar Matrix

```

4. Unit Matrix (Identity Matrix or primary Diagonal Unit Matrix):

A diagonal matrix where all the elements on the main diagonal (primary diagonal) are 1.

Example:
$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

```

#include <stdio.h>

int main()
{
    int row, col;
    scanf("%d %d", &row, &col);

    int arr[row][col];
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            scanf("%d", &arr[i][j]);
        }
    }

    int flag = 1;
    // check if the matrix is square matrix or not
    if (row != col)
    {
        flag = 0;
    }
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            if (i == j)
            {

```

```

        if (arr[i][j] != 1)
        {
            flag = 0;
        }
    }
    else if (arr[i][j] != 0)
    {
        flag = 0;
    }
}

if (flag == 1)
{
    printf("Unit Matrix");
}
else
{
    printf("Not unit Matrix");
}

return 0;
}

```

```

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
3 3
1 0 0
0 1 0
0 0 1
Unit Matrix

```

Secondary Diagonal Unit Matrix:

A Diagonal Matrix where all the elements on the secondary diagonal are 1

Example: $\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$

```

#include <stdio.h>

int main()
{
    int row, col;
    scanf("%d %d", &row, &col);

    int arr[row][col];
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)

```

```

    {
        scanf("%d", &arr[i][j]);
    }
}

int flag = 1;
// check if the matrix is square matrix or not
if (row != col)
{
    flag = 0;
}
for (int i = 0; i < row; i++)
{
    for (int j = 0; j < col; j++)
    {
        if (i + j == row - 1)
        {
            if (arr[i][j] != 1)
            {
                flag = 0;
            }
        }
        else if (arr[i][j] != 0)
        {
            flag = 0;
        }
    }
}

if (flag == 1)
{
    printf("Secondary unit Matrix");
}
else
{
    printf("Not secondary unit Matrix");
}

return 0;
}

```

```

conta@Tamim MINGW64 ~/Downloads/C Programming
$ ./hello
3 3
0 0 1
0 1 0
1 0 0
Secondary unit Matrix

```

Chapter 6: introduction to string

6.1 what is string:

String is a character array.

Program-1:

```
#include <stdio.h>

int main(){

    char str[5] = {'t','a','m','i','m'};

    printf("%c%c%c%c%c", str[0], str[1], str[2],str[3],str[4]);

    return 0;

}
```

Program-2:

```
#include <stdio.h>

int main(){

    char str[5] = {'t','a','m','i','m'};

    for(int i = 0; i < 5; i++){

        printf("%c", str[i]);

    }

    return 0;

}
```

Program-3:

```
#include <stdio.h>

int main(){

    char str[5];

    for(int i = 0; i < 5; i++){

        scanf("%c", &str[i]);

    }

    for(int i = 0; i < 5; i++){

        printf("%c", str[i]);

    }

}
```

Sizeof():

Program -1:

```
#include <stdio.h>

int main(){

    printf("%d\n", sizeof(int));

    printf("%d\n", sizeof(long long int));

    printf("%d\n", sizeof(float));

    printf("%d\n", sizeof(double));

    printf("%d\n", sizeof(char));


    return 0;

}
```

Program-2:

```
#include <stdio.h>

int main(){

    int arr[5];

    char str[5];

    printf("%d\n", sizeof(arr));

    printf("%d", sizeof(str));


    return 0;

}
```

Program-3:

```
#include <stdio.h>

int main(){

    int arr[5];

    char str[5];

    printf("%d\n", sizeof(arr) / sizeof(int));

    printf("%d", sizeof(str)/sizeof(char));

    return 0;

}
```

6.2 string input and output:

Program 1:

```
#include <stdio.h>

int main(){

    char str[5] = "tamim"; // first special power

    for(int i = 0; i < 5; i++){

        printf("%c", str[i]);

    }

    return 0;

}
```

Program 2:

Error:

```
#include <stdio.h>

int main(){

    char str[5] = "tamim";

    printf("%s", str);

    return 0;

}
```

Solution:

```
#include<stdio.h>

int main(){

    char str[6] = "tamim\0";

    printf("%s", str); // second special power

}
```

OR

```
#include <stdio.h>

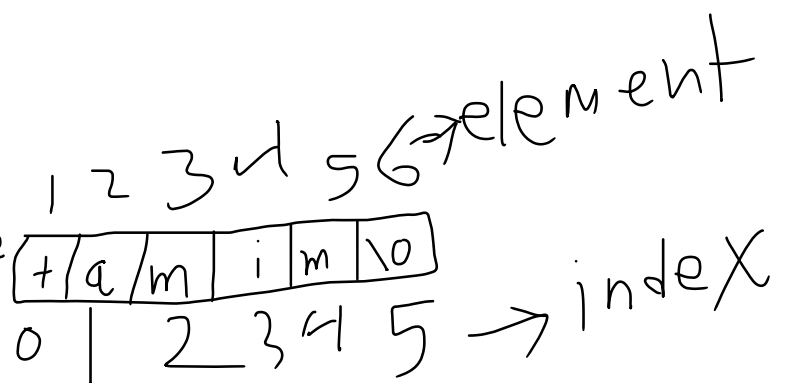
int main(){

    char str[6] = "tamim"; // third power

    printf("%s", str);

    return 0;

}
```



Program 3:

```
#include <stdio.h>

int main(){
    char str[6];
    scanf("%s", str);
    printf("%s", str);
}
```

Note: This code has a problem. This code doesn't take space as an input. Here scanf() count space as her ending.

Solution-1:

```
#include <stdio.h>

int main(){
    char str[12];
    gets(str);
    printf("%s", str);
}
```

Note: gets() reads an entire line of input until Enter is pressed, without checking the size of the buffer, which can lead to overflow. fgets() is safer as it reads up to N - 1 characters or until Enter is pressed, preventing buffer overflow. We should always use fgets() instead of gets().

Solution-2:

```
#include <stdio.h>

int main(){
    char str[12];
    fgets(str,12,stdin);
    printf("%s", str);
}
```

NOTE: gets() and fgets() both accept spaces in the input, but scanf() does not. When you press Enter, both gets() and fgets() stop reading input, but fgets() includes an extra newline (\n) after pressing Enter.

Program 4:

gets() -- puts() = gets() reads a string from the user (unsafe), and puts() prints a string by adding an additional newline.

```
#include <stdio.h>

int main() {
    char str[100];
    printf("Enter a string: ");
    gets(str);
    puts("You entered: ");
    puts(str);
    return 0;
}
```

Program 5:

fgets() -- fputs() = fgets() reads a string from the user safely, and fputs() prints a string without adding a newline.

```
#include <stdio.h>

int main() {
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin); // Safely reading a string
    fputs("You entered: ", stdout); // Printing the prompt
    fputs(str, stdout); // Printing the string without an automatic newline
    return 0;
}
```

6.3 Length of a string:

1	2	3	4	5	
t	a	m	i	m	\0

Length = 5

Solution 1:

```
#include <stdio.h>
```

```
int main(){
```

```
    char str[100];
```

```
    scanf("%s", str);
```

```
//scanf("%s ", str); error
```

```
    int i = 0;
```

```
    int count = 0;
```

```
    while(str[i] != '\0'){
```

```
        count++;
```

```
        i++;
```

```
    }
```

```
    printf("%d", count);
```

```
    return 0;
```

```
}
```

Or

```
#include<stdio.h>
```

```
int main(){
```

```
    char str[100];
```

```
    scanf("%s", str);
```

```
    int count = 0;
```

```
    for(int i = 0; str[i] != '\0'; i++){
```

```
        count++;
```

```
    }
```

```
    printf("%d", count);
```

```
}
```

Solution 2: With built-in function

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main(){
```

```
    char str[100];
```

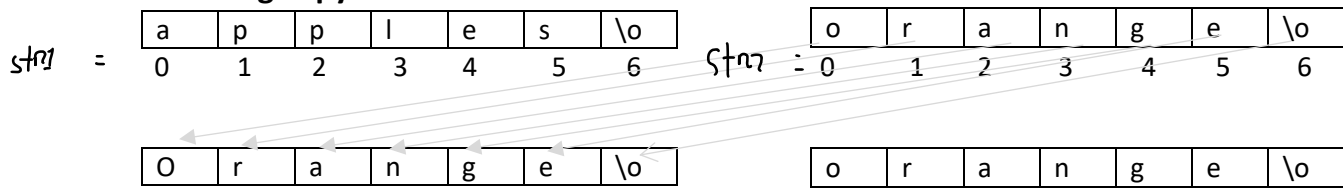
```
    scanf("%s", str);
```

```
    printf("%d", strlen(str));
```

```
    return 0;
```

```
}
```

6.4 string copy:



Method 1: Without built-in function

```
#include <stdio.h>

#include <string.h>

int main(){

    char str1[100], str2[100];

    scanf("%s %s", str1, str2);

    for(int i = 0; i <= strlen(str2); i++){

        str1[i] = str2[i];

    }

    printf("%s %s", str1, str2);

}
```

Method 2: with built-in function

```
#include <stdio.h>

#include <string.h>

int main(){

    char str1[100], str2[100];

    scanf("%s %s", str1, str2);

    strcpy(str1, str2);

    printf("%s %s", str1, str2);

}
```

6.5 string lexicographical comparison:

apple < apply

apple < applicant

acde > abcdegdf

abcd > abc

abd < abde

abd == abd

abc < edf

abcd < adef

so here we get 4 cases:

Case 1: if both strings have reached their end (\0) at the same time it means they are the same.

Case 2: If str1 ends first (reaches \0 before str2), it means that str1 is shorter than str2

Case 3: If str2 ends first (reaches \0 before str1), it means that str2 is shorter than str1.

Case 4: If the characters at the current position are the same, we move to the next character by incrementing i.

Case 5: If str1 is less than str2, it means that str1 is shorter than str2

Case 6: If str2 is less than str1, it means that str2 is shorter than str1

pseudo code:

str1[i] == '\0' && str2[i] == '\0' ---> str1 and str2 are same

str1[i] == '\0' ---> str1 is small

str2[i] == '\0' ---> str2 is small

str1[i] == str2[i] ---> i++

str1[i] < str2[i] ---> str1 is small

str2[i] < str1[i] ---> str2 is small

Without built-in function:

```
#include <stdio.h>
```

```
int main(){
```

```
    char str1[100], str2[100];
```

```
    scanf("%s %s", str1, str2);
```

```
    int i = 0;
```

```
while(1){  
    // Case 1  
    if(str1[i] == '\0' && str2[i] == '\0'){  
        printf("str1 == str2");  
        break;  
    }  
  
    // Case 2  
    else if(str1[i] == '\0'){  
        printf("str1 is smaller");  
        break;  
    }  
    // case 3  
    else if(str2[i] == '\0'){  
        printf("str2 is smaller");  
        break;  
    }  
  
    // case 4  
    if(str1[i] == str2[i]){  
        i++;  
    }  
    // case 5  
    else if(str1[i] < str2[i]){  
        printf("str1 is smaller");  
        break;  
    }  
    // case 6  
    else if(str2[i] < str1[i]){  
        printf("str2 is smaller");  
        break;  
    }  
}
```

```

    }
}
return 0;
}

```

With built-in function:

```

#include <stdio.h>
#include <string.h>
int main(){
    char str1[100], str2[200];
    scanf("%s %s", str1, str2);

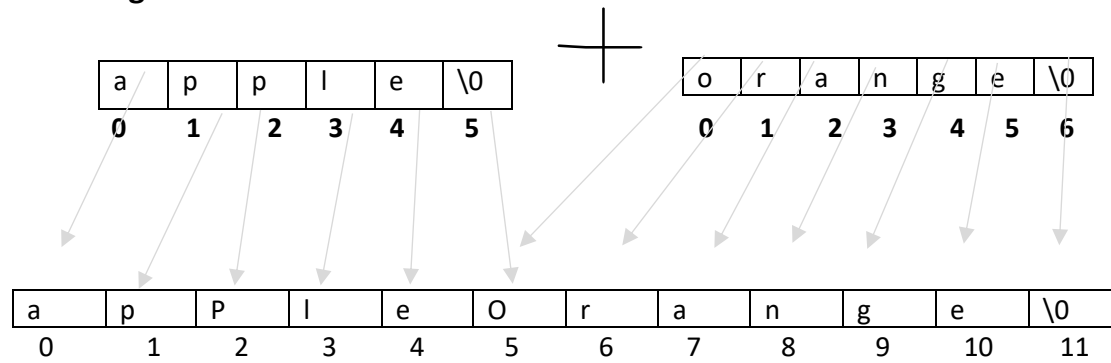
    int check = strcmp(str1, str2);
    if(check < 0){
        printf("str1 is small");
    }
    else if(check > 0){
        printf("str2 is small");
    }
    else{
        printf("Str1 == str2");
    }
    return 0;
}

```

Note: here strcmp(str1,str2) function return 3 values:

1. Value of strcmp == 0 ---> str1 == str2
2. Value of strcmp < 0 ---> str1 is small
3. Value of strcmp > 0 ---> str2 is small

6.6 string concatenation:



Without built-in function:

```
#include <stdio.h>
#include <string.h>

int main(){
    char str1[100], str2[100];

    scanf("%s %s", str1, str2);

    char str[strlen(str1) + strlen(str2)];

    for(int i = 0; i < strlen(str1); i++){
        str[i] = str1[i];
    }
    for(int i = 0; i <= strlen(str2); i++){
        str[strlen(str1) + i] = str2[i];
    }

    printf("%s", str);
    return 0;
}
```

or

```
#include <stdio.h>
#include <string.h>
int main(){
    char a[200], b[100];
    scanf("%s %s", a,b);
    int len_a = strlen(a);
    for(int i = 0; i <= strlen(b); i++){
        a[len_a] = b[i];
        len_a++;
    }
    printf("%s", a);
    return 0;
}
```

With built-in function:

```
#include <stdio.h>
#include <string.h>
int main(){
    char a[200], b[100];
    scanf("%s %s", a,b);
    strcat(a,b);
    printf("%s", a); // printf("%s", strcat(a, b));
    return 0;
}
```

6.7 counting or frequency String (Problems):

```

C:\Users\conta\OneDrive\Des
abacz
a = 2
b = 1
c = 1
d = 0
e = 0
f = 0
g = 0
h = 0
i = 0
j = 0
k = 0
l = 0
m = 0
n = 0
o = 0
p = 0
q = 0
r = 0
s = 0
t = 0
u = 0
v = 0
w = 0
x = 0
y = 0
z = 1

Process returned 0 (0x0)   execution time : 10.824 s
Press any key to continue.

```

Solution-1:

```

#include <stdio.h>

#include <string.h>

int main(){

    char str[100];

    scanf("%s", str);

    int cnt[26] = {0};

    for(int i = 0; i < strlen(str); i++){

        cnt[str[i] - 'a']++; // or str[i] - 97;

    }

    for(int i = 0; i < 26; i++){

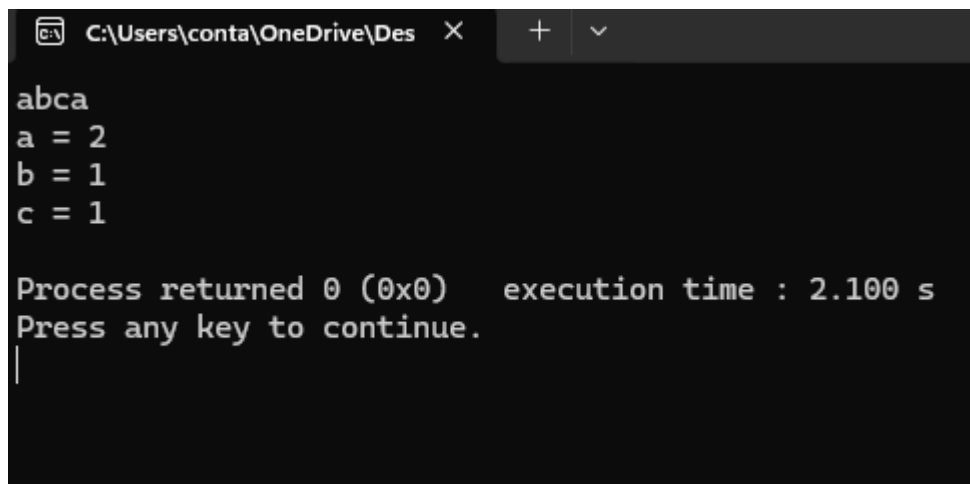
        printf("%c = %d\n", i + 'a', cnt[i]);

    }

    return 0;

}

```

Solution 2:

```
C:\Users\conta\OneDrive\Des X + v
abca
a = 2
b = 1
c = 1

Process returned 0 (0x0) execution time : 2.100 s
Press any key to continue.
|
```

```
#include <stdio.h>
#include <string.h>
int main(){
    char str[100];
    scanf("%s", str);
    int cnt[26] = {0};

    for(int i = 0; i < strlen(str); i++){
        int value = str[i] - 'a';
        cnt[value]++;
    }
    for(int i = 0; i < 26; i++){
        if(cnt[i] != 0){
            printf("%c = %d\n", i + 97, cnt[i]);
        }
    }

    return 0;
}
```

Solution-3:

```

C:\Users\conta\OneDrive\Des
abcazdea
a = 3
b = 1
c = 1
z = 1
d = 1
e = 1

Process returned 0 (0x0)   execution time : 6.537 s
Press any key to continue.
|

```

```

#include <stdio.h>
#include <string.h>
int main(){
    char str[100];
    scanf("%s", str);
    int cnt[26] = {0};

    for(int i = 0; i < strlen(str); i++){
        int value = str[i] - 'a';
        cnt[value]++;
    }
    for(int i = 0; i < strlen(str); i++){
        int value = str[i] - 97;
        if(cnt[value] != 0){
            printf("%c = %d\n", value + 97, cnt[value]);
        }
        cnt[value] = 0;
    }
    return 0;
}

```

Chapter 7: Function

7.1 what is function:

A Function is a block of code that perform a specific task when it is called.

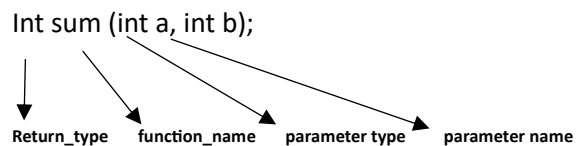
Function declaration:

Syntax:

Return_type function_name (parameters);

Code:

Int sum (int a, int b);



Return_type function_name parameter type parameter name

Note: Function Return type == data types or void. void means the function return nothing.

Function definition:

Syntax:

```
Return_type function_name (parameters) {
    Function body
}
```

Code:

```
Int sum (int a, int b) {
    Int result = a + b;
    Return result;
}
```

Function call:

Syntax:

Function_name (arguments);

Code:

```
int value = sum(10,20);
```

now function declaration + definition + call =

```
#include <stdio.h>
```

```
int sum (int a, int b) {
    int result = a + b;
    return result;
}

int main(){
    int value = sum (10,20);
    printf("%d", value);
}
```

Function Declaration

Function definition

Function calling

● = works 1st

● = works 2nd

● = works 3rd (checks that the result value and the return type same or not. If both match then when the function call, the return value goes there.)

● = works 4th

Note: here a, b are parameters and 10, 20 are arguments

There are two different write a function:

Way 1:

Example-1:

```
#include <stdio.h>
```

```
int dbl(int x){
```

```
    int s = x * 2;
```

```
    return s;
```

```
}
```

```
int main(){
```

```
    int f = dbl(1500);
```

```
    printf("%d", f);
```

```
}
```

Example-2:

```
#include <stdio.h>

int areaOfRect(int length, int width){
    int area = length * width;
    return area;
}

int main(){
    int l = 10, w = 5;
    int area = areaOfRect(l,w);
    printf("%d\n", area);

    l = 10, w = 10;
    area = areaOfRect(l,w);
    printf("%d", area);
}
```

Way- 2

Example- 1:

```
#include <stdio.h>

int dbl(int x);

int main(){
    int f = dbl(1500);
    printf("%d", f);
}

int dbl(int x){
    int s = x * 2;
    return s;
}
```

Example - 2:

```
#include <stdio.h>

int areaOfRect(int length, int width);

int main(){
    int l = 10, w = 5;

    int area = areaOfRect(l,w);

    printf("%d\n", area);

    l = 10, w = 10;

    area = areaOfRect(l,w);

    printf("%d", area);
}

int areaOfRect(int length, int width){
    int area = length * width;

    return area;
}
```

7.2 return + parameter:

Example- 1:

```
#include <stdio.h>

int sum(int x, int y){
    int result = x + y;

    return result;
}

int main(){
    printf("%d", sum(10, 20));
}
```

Example-2:

```
#include <stdio.h>

int sum(int x, int y){
    int result = x + y;

    return result;
}
```

```

}

int main(){

    int x,y;

    // main function x and y != sum function x and y

    scanf("%d %d", &x, &y);

    printf("%d", sum(x,y));

}

```

7.3 return + no parameter:

Note: If we want no parameter then we have two options one is to write void and another is to write nothing. If we use void in the parameter and forcefully gives value on it, then it will show error. But if we write nothing in the parameter and forcefully gives value on it, then it will don't show error.

Function with `void` in the parameter list:

```

#include <stdio.h>

int calculateNumber(void) {

    int a = 5, b = 5;

    int result = a + b;

    return result;

}

int main() {

    int number = calculateNumber();

    printf("The calculated number is: %d\n", number);

    return 0;

}

```

Function with an empty parameter list:

Example- 1:

```

#include <stdio.h>

int calculateNumber() {

    int a = 5, b = 5;

    int result = a + b;

    return result;

}

int main() {

```

```

    int number = calculateNumber();

    // Call the function and receive the return value
    printf("The calculated number is: %d\n", number);

    return 0;
}

```

Example-2:

```

#include <stdio.h>

int sum(){
    int a, b;

    scanf("%d %d", &a, &b);

    int result = a + b;

    return result;
}

int main(){
    int result = sum();

    printf("%d", result);

    return 0;
}

```

Example-3:

```

#include <stdio.h>

int sum(){
    printf("Bisshash koro input nicchi\n");

    return 0;
}

int main(){
    printf("Ami kintu call kortesi\n");

    sum();

    printf("dekhi ki korla tumi\n");

    return 0;
}

```

7.4 No return + parameter:

```
#include <stdio.h>
```

```
void sum(int x, int y){
```

```
    int result = x + y;
```

```
    printf("%d\n", result);
```

```
}
```

```
int main(){
```

```
    int a, b;
```

```
    scanf("%d %d", &a, &b);
```

```
    sum(a,b);
```

```
    return 0;
```

```
}
```

7.5 no return + no parameter:

```
#include <stdio.h>
```

```
void sum(void){
```

```
    int a, b;
```

```
    scanf("%d %d", &a, &b);
```

```
    int result = a + b;
```

```
    printf("%d\n", result);
```

```
}
```

```
int main(){
```

```
    sum();
```

```
    sum();
```

```
}
```

7.6 useful built-in function:

Math.h:

```
#include<stdio.h>

#include <math.h>

int main(){

printf("ceil = %f",ceil(3.6));

//ceil = It returns the integer value which is greater than or equal to given number.

printf("\nfloor = %f",floor(3.6));

// floor = It returns the integer value which is less than or equal to given number.

printf("\nround =%f",round(3.6));

printf("\nround = %f",round(3.5));

printf("\nround = %f",round(3.4));

// round = It returns the integer value which is less than or equal to given number if the number is
less then .5 and It returns the integer value which is greater than or equal to given number if the
number is equal or greater than .5

printf("\nsqrt = %f",sqrt(16));

// sqrt = returns the square root of given number.

printf("\npow %f",pow(2,4));

// pow(base, exponent) = returns the power of given number.

printf("\nabs = %d",abs(-12));

// <stdlib.h>

// returns the absolute value of given number. That's means it convert (-) to (+)

return 0;

}
```

7.7 scopes in c:

The scope of a variable in C is the block or the region in the program where a variable is declared, defined, and used. Outside this region, we cannot access the variable and it is treated as an undeclared identifier.

Example-1:

```
// C program to illustrate the scope of a variable
#include <stdio.h>

void func();

int main()
{
    /*Scope of this variable is within main() function. */
    int var = 34;

    printf("%d", var);
    return 0;
}

// function where we try to access the var defined in main()
void func() {
    printf("%d", var);
}
```

Example-2:

```
// C program to illustrate the global scope
#include <stdio.h>

/*variable declared in global scope. global variable accessed from anywhere*/
int global = 5;

void display()
{
    printf("%d\n", global);
}
```

```

int main()
{
    printf("Before change within main: ");
    display();

    /*changing value of global variable from main function */
    printf("After change within main: ");
    global = 10;
    display();
}

```

Example-3:

// C program to illustrate the local scope

```
#include <stdio.h>
```

```

int main()
{
    {
        int x = 10, y = 20;
        {
            printf("x = %d, y = %d\n", x, y);
            {
                int y = 40;
                x++;
                y++;
                printf("x = %d, y = %d\n", x, y);
            }
            printf("x = %d, y = %d\n", x, y);
        }
    }
    return 0;
}

```

Chapter 8: Recursion

8.1 call Stack:

The call stack is a special region of your computer's memory that stores information about the active functions of a computer program. It's used to keep track of where each function is in its execution and what it needs to return to once it's done. It follows a Last-In-First-Out (LIFO) structure.

```
#include <stdio.h>
```

```
void functionA() {  
    int a = 10;  
    printf("Inside functionA: a = %d\n", a);  
}
```

```
void functionB() {  
    int b = 20;  
    functionA();  
    printf("Inside functionB: b = %d\n", b);  
}
```

```
int main() {  
    int c = 30;  
    functionB();  
    printf("Inside main: c = %d\n", c);  
    return 0;  
}
```

Visualizing the Call Stack:

1. Before any function is called:

[Empty]

2. When main is running:

[main]

3. When functionB is called from main:

[functionB]

[main]

4. When functionA is called from functionB:

[functionA]

[functionB]

[main]

5. After functionA returns to functionB:

[functionB]

[main]

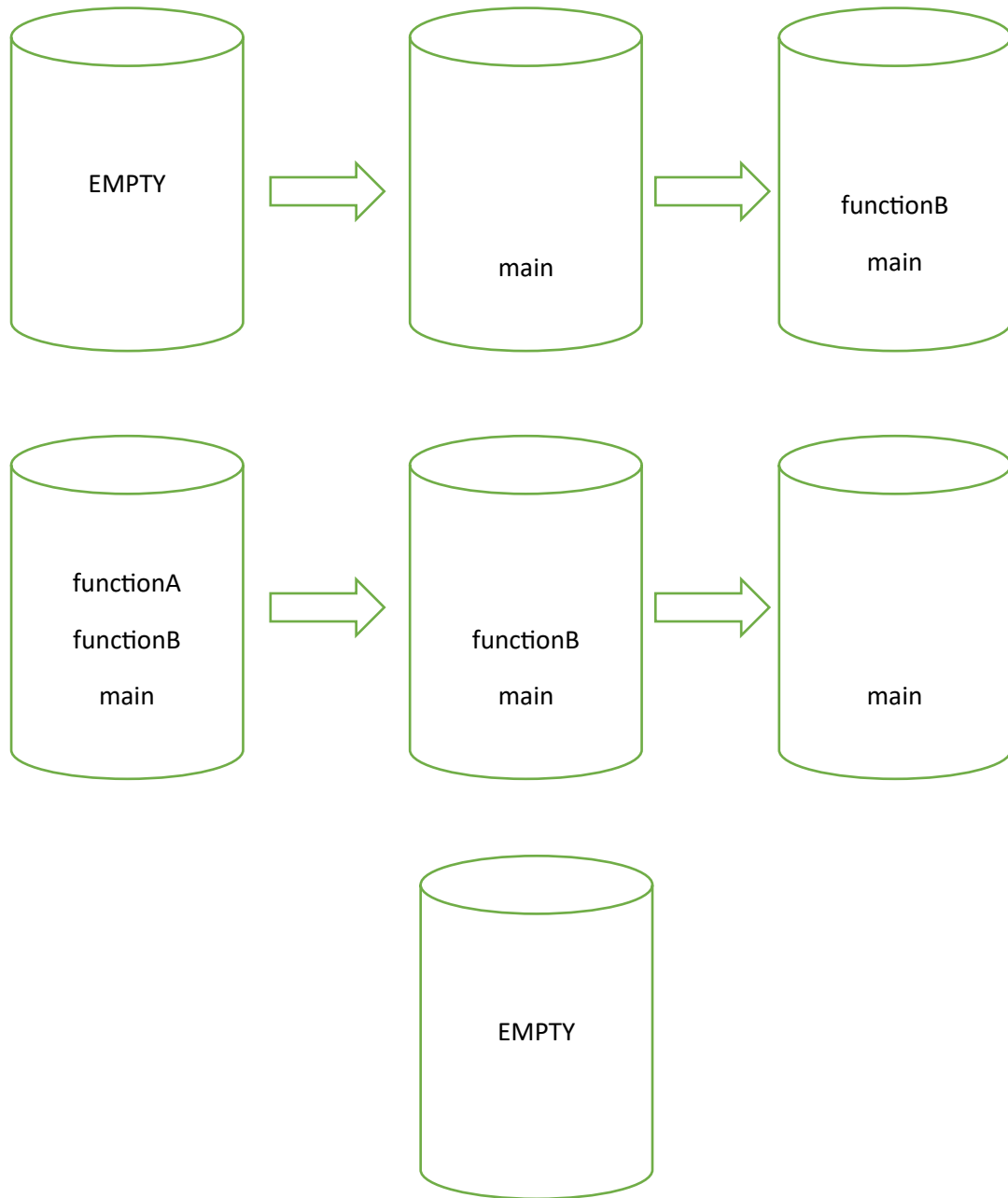
6. After functionB returns to main:

[main]

7. After main completes:

[Empty]

Call Stack



8.2 What is Recursion:

Recursion in programming is a technique where a function calls itself in order to solve a problem. A recursive function typically has two main components:

Base Case: The base case is a condition in a recursive function that stops the recursion. It ensures that the function does not call itself indefinitely, which prevents a stack overflow.

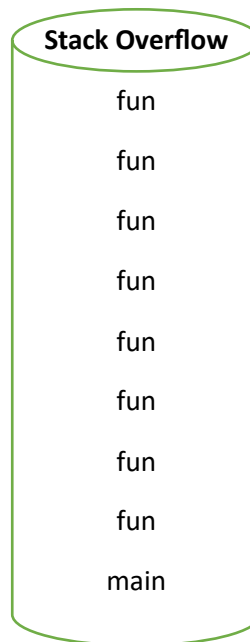
Recursive Case: This is the part of the function where the function calls itself with modified arguments, moving towards the base case.

```
#include <stdio.h>
```

```
void fun() {
    printf("Hello\n");
    fun();
}

int main() {
    fun();
    return 0;
}
```

```
// Error: Stack Overflow
```

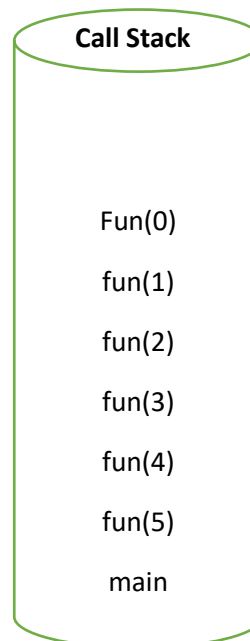


To prevent infinite recursion and a stack overflow, you need to add a base case and recursive case to the recursive function.

```
#include <stdio.h>
```

```
void fun(int count) {
    if (count == 0) { // Base Case
        return;
    }
    printf("Hello\n");
    fun(count - 1); // Recursive Case
}

int main() {
    fun(5); // Initial Call with a limit
    return 0;
}
```



8.3 Print 1 to 5 using recursion:

```
#include<stdio.h>

void fun(int i, int n){
    if(i == n + 1)
        return;
    printf("%d\n", i);
    fun(i+1, n);
}

int main(){
    int n;
    scanf("%d", &n);
    fun(1,n);
    return 0;
}
```

8.4 Print 5 to 1 using recursion:

Example-1:

```
#include<stdio.h>

void fun(int i, int n){
    if(i == n - 1)
        return;
    printf("%d\n", i);
    fun(i-1, n);
}

int main(){
    int n;
    scanf("%d", &n);
    fun(5,n);
    return 0;
}
```

Example-2:

```
#include<stdio.h>

void fun(int i){
    if(i == 0){
        return;
    }
    printf("%d\n", i);
    fun(i - 1);
}

int main(){
    fun(5);
    return 0;
}
```

Example-3:

```
#include<stdio.h>

void fun(int i){
    if(i == 6)
        return;
    fun(i + 1);
    printf("%d\n", i);
}

int main(){
    fun(1);
    return 0;
}
```

8.5 printing array using recursion:

```
#include<stdio.h>

void fun(int a[], int n, int i){
    if(i == n){
        return;
    }
}
```

```

    printf("%d\n", a[i]);
    fun(a, n, i+1);
}
int main(){
    int n;
    scanf("%d", &n);
    int a[n];
    for(int i = 0; i < n; i++){
        scanf("%d", &a[i]);
    }
    fun(a,n,0);
    return 0;
}

```

8.6 Length of a string using recursion:

```
#include<stdio.h>
```

```

int fun(char a[], int i){
    if(a[i] == '\0')
        return 0;
    int length = fun(a, i + 1);
    return length + 1;
}
int main(){
    char a[20] = "Tamim";
    int length = fun(a, 0);
    printf("%d\n", length);
    return 0;
}

```

Chapter 9: Pointer

9.1 Pointers:

The pointer is a variable which stores the address of another variable.

Declaring a pointer:

Int *ptr;  Asterisk symbol

Initialization a pointer:

```
int var = 10;
```

```
int *ptr = &var;
```

Assignment a pointer:

```
*ptr = 20
```

Example-1:

```
#include<stdio.h>
```

```
int main(){
```

```
    int x = 10;
```

```
    printf("Address = %p\n", &x); // print the address of x variable
```

```
    int *ptr = &x; // Now the address of x variable is stored to the pointer variable ptr
```

```
    printf("Address = %p\n", ptr); // print the address of pointer ptr variable
```

```
    return 0;
```

```
}
```

Example 2:

```
#include<stdio.h>
```

```
int main(){
```

```
    int x = 100;
```

```
    int *ptr = &x;
```

```
    printf("Address of x = %p\n", &x);
```

```
    printf("Value of ptr = %p\n", ptr);
```

```
    printf("Addres of ptr = %p\n", &ptr);
```

```
    return 0; }
```

Example-3:

```
#include<stdio.h>

int main(){
    int x = 10;

    printf("Value of x variable = %d\n", x);
    printf("Address of x variable = %p\n", &x);

    int *ptr = &x;
    printf("Value of ptr pointer variable = %d\n\n", *ptr);

    *ptr = 20; // update the ptr variable
    printf("Value of x variable = %d\n", x);
    printf("Value of ptr pointer variable = %d\n", *ptr);
    printf("Address of x variable = %p\n", &x);
    printf("Address of Ptr pointer variable = %p\n", ptr);
    return 0;
}
```

Example-4: Dereferencing a pointer

Dereferencing a pointer means accessing or modifying the value that the pointer points to.

```
#include <stdio.h>

int main() {
    int x = 10; // initialize a variable
    int *ptr = &x; // initialize a pointer
    printf("Value at ptr: %d\n", *ptr); // Dereferencing : Dereference the pointer to access value of x

    *ptr = 20; //Dereferencing: Dereference the pointer to modify value
    printf("New value of x: %d\n", x);

    return 0;
}
```

9.2 Call by Value or pass by value:

Call by value is a method of passing arguments to a function in which the actual value of the argument is passed. This means that any modifications made to the parameter inside the function do not affect the original value of the argument.

```
#include <stdio.h>
```

```
void fun(int x){
```

```
    x = 100;
```

```
}
```

```
int main(){
```

```
    int x = 10;
```

```
    fun(x);
```

```
    printf("%d\n", x);
```

```
}
```

9.3 Call by reference or passed by reference (Pointer Dereferencing Technique):

Call by reference is a method of passing arguments to a function where the actual memory address of the argument is passed, allowing the function to modify the original variable. Changes made to the parameter affect the passed argument because they refer to the same memory location.

```
#include<stdio.h>
```

```
void fun(int *p){
```

```
    *p = 100;
```

```
}
```

```
int main(){
```

```
    int x = 10;
```

```
    fun(&x);
```

```
    printf("Value of x = %d", x);
```

```
}
```

9.4 Relation Between array and pointer:

When we declare an array, the array name actually acts as a pointer because the array name stores the address of the first index. If we want to see proof:

Example-1:

```
/*
#include <stdio.h>

int main() {
int number = 42;
int *ptr = &number;

// Printing memory address using %p
printf("Memory address of 'number': %p\n", &number);
// Printing pointer value using %p
printf("Value of the pointer 'ptr': %p\n", ptr);

return 0;
}

*/
#include<stdio.h>
int main(){
int arr[5] = {10,20,30,40,50};

printf("Printing memory address using %%p = %p\n", &arr);
printf("Printing memory address using %%p = %p\n", &arr[0]);

printf("Printing pinter value using %%p = %p\n", arr); /* That means arr is a pointer and by default
arr holding arr[0] memory address */

return 0;
}
```

Example-2:

```
#include<stdio.h>

int main(){
    int arr[5] = {10,20,30,40,50};

    printf("value of arr[0] index = %d\n", arr[0]);
    printf("value of arr pointer = %d\n", *arr);// Dereferencing
    // So this is a another prove that arr is a pointer.
}
```

How to get all of the array element using pointer dereferencing technique:

```
#include<stdio.h>

int main(){
    int arr[5] = {10,20,30,40,50};
    printf("Value of First element = %d\n", arr[0]);
    printf("Value of First element = %d\n", 0[arr]);
    printf("Value of First Element using Dereferencing = %d\n", *arr);
    printf("Value of First Element using Dereferencing = %d\n", *(arr+0)); // arr[0] == *(arr + 0)
    printf("Value of First Element using Dereferencing = %d\n", *(0+arr)); // 0[arr] == *(0 + arr)
    printf("Value of Second Element using Dereferencing = %d\n", *(arr+1));
    printf("Value of Third Element using Dereferencing = %d\n", *(arr+2));
    printf("Value of Fourth Element using Dereferencing = %d\n", *(arr+3));
    printf("Value of Fifth Element using Dereferencing = %d\n", *(arr+4));
}
```

9.5 How to pass array into a function:

When you pass an array to a function, what actually gets passed is a pointer to the first element of the array, not the entire array itself.

Example: Find array size in functions

```
#include<stdio.h>

void fun(int arr[]){

    int size = sizeof(arr) / sizeof(int); // 8 / 4 = 2

    printf("Array size in the fun function = %d\n", size);

}

int main(){

    int arr[5] = {10,20,30,40,50};

    int size = sizeof(arr) / sizeof(int); // 20 / 4 = 5

    printf("Array size in the main function = %d\n", size);

    fun(arr);

}
```

So, in this example, the array size in the main function = 5, and the array size in the fun function = 2 (incorrect). This means the fun function doesn't know how many elements are in the array. That's why you need to pass the size of the array separately if you want to work with the entire array inside the function.

Example: Passing array into function

```
#include<stdio.h>

void fun(int arr[], int n){ // int arr[] == int *arr

    for(int i = 0; i < n; i++){

        printf("%d ", arr[i]);

    }

}

int main(){

    int arr[5] = {10,20,30,40,50};

    fun(arr,5);

    return 0;

}
```

Modifying Array Elements:

```
#include <stdio.h>

void func(int a[], int n) {
    a[0] = 100; // Change the value of the first element
}

int main() {
    int a[5] = {10, 20, 30, 40, 50};
    func(a, 5); // Pass the array and its size to func
    for (int i = 0; i < 5; i++) {
        printf("%d ", a[i]); // Print the elements of the array
    }
    return 0;
}
```

9.6 How to pass string into a function:

Strings in C are always terminated with a null character (\0), which marks the end of the string. When passing strings to functions in C, you don't need to pass the length of the string. This is because:

1. **strlen() Function:** The strlen() function calculates the string length by counting characters until it encounters the null terminator (\0). This means you don't need to pass the length separately for finding the length of the string.
2. **%s Format Specifier:** The %s format specifier in printf function prints characters from the string starting at the given pointer and stops when it reaches the null terminator (\0). This allows the function to handle the string correctly without needing its length and a loop.

Example: Find string length in functions

```
#include <stdio.h>
#include <string.h>

void fun(char s[]) {
    int sz = strlen(s);
    printf("%d\n", sz);
}
```

```

int main() {
    char s[10] = "Hello";
    fun(s);
    int sz = strlen(s);
    printf("%d\n", sz);
    return 0;
}

```

Example: Passing string into function

```
#include <stdio.h>
```

```

void printString(char *str) { // char *str == char str[]
    printf("The string is: %s\n", str);
}

```

```

int main() {
    char myString[] = "Hello, World!";
    printString(myString);
    return 0;
}

```

Modifying string Elements:

```

#include<stdio.h>

void fun(char *arr){
    arr[0] = 'G';
}

int main(){
    char arr[]="hello";
    fun(arr);
    printf("%s", arr);
    return 0;
}

```